What Moves the Number? Evaluation and Encoder Choices Dominate Architecture in Vietnamese Spam-Review Classification

Phan Ba Song Toan, Trinh Minh Bao Tran, and Tran Quang Long
Da Nang 550000, Vietnam

***Abstract*—Vietnamese e-commerce reviews are labelled as non-spam, fake review, brand-only review or non-review, and a PhoBERT classifier already reaches the mid seventies in macro-F1 on this four-class task. Papers in this area usually report one number on one split and attribute the gain to the architecture. We test that attribution directly. Under one locked training protocol we run 53 configurations and measure how much each design choice moves macro-F1: a hierarchical multi-task objective, metadata fusion that encodes the product description in a separate pass, the backbone, the token window, the learning-rate schedule, and two evaluation choices — removing training text that also occurs verbatim in the test split, and holding products out of training. The architecture choices move the metric by at most a tenth of a point. The evaluation choices move it by 1.4 to 2.3 points and reverse the ranking of the two models, and changing the backbone costs 1.5. Seed noise alone is 0.7 to 1.0. We also report an error decomposition showing that missed spam rather than wrong spam type dominates the remaining loss, and a quantified share of label noise in template-like non-spam reviews.**

***Index Terms*—spam review detection, Vietnamese NLP, PhoBERT, multi-task learning, product-disjoint evaluation, reproducibility**

I. Introduction

Reviews drive product discovery on e-commerce platforms, and the same incentive that makes them valuable also makes them a target: sellers can post promotional text, competitors can post damaging text, and both can be automated. Vietnamese platforms add a further difficulty. User text is short, informal, full of abbreviations and teencode, mixed with emoji, and often copied verbatim between products, so a classifier can reach a respectable score by memorising templates rather than by judging content.

The ViSpamReviews dataset [1] and its metadata-extended version [2] formalise the problem as two tasks. Task 1 is binary: non-spam versus spam. Task 2 is four-class: non-spam plus three spam types. Prior work reports PhoBERT [8] as the strongest text-only backbone, with metadata fusion improving results further [2]. Two properties of that evaluation limit how far the numbers can be generalised. First, the split is random over reviews, so the same product can appear in training and test data; product-specific wording and copied templates then inflate the score. Second, the four classes are not independent: a review cannot be both non-spam and a spam type, so a flat four-way classifier ignores the structure that the label set already encodes.

This paper is deliberately a small, auditable study rather than a leaderboard entry. We ask four questions and answer each with artifacts that can be re-run:

1) **Reproduction.** Can the published PhoBERT result be re-obtained from the public release, and where do the public and internal dataset snapshots diverge?
2) **Supervision.** Does a hierarchical two-head objective, which trains a binary detector and a spam-type classifier on a shared encoder, improve the four-class metric when both models share one protocol and are evaluated over several seeds?
3) **Generalisation.** How much accuracy depends on seeing the same product during training, and does text-level duplication remain a leak even after product-level splitting?
4) **Sensitivity.** Of all these choices, which ones actually move the metric, and are the effects larger than seed noise?

Our contributions are: (i) an effect-size table that puts every design choice on one scale, with seed noise as the ruler, and shows that evaluation choices outweigh architectural ones by an order of magnitude; (ii) a documented, single-protocol comparison in which the only difference between the two models is the supervision objective, with three seeds, paired bootstrap intervals and an exact McNemar test; (iii) a product-disjoint split with an explicit leakage audit, extended by a verbatim-duplicate audit that the original evaluation does not report; (iv) a quantitative decomposition of the remaining errors, separating spam detection from spam typing, plus a measured share of label noise in template-like non-spam reviews; and (v) negative results for class-weighted supervision, naive metadata concatenation and a fusion baseline that avoids truncation altogether. We also state plainly which of our earlier exploratory numbers are *not* comparable with the ones reported here, because they were produced before the protocol was locked.

II. Related Work

A. Opinion spam and review spam detection

Jindal and Liu [3] introduced opinion spam as a text-classification problem and showed that duplicate and near-duplicate reviews are a strong signal. Ott et al. [4] showed that crowd workers can be induced to write deceptive reviews and that n-gram classifiers separate them from genuine ones almost perfectly, a result that later work attributed partly to the construction of the positive class. Mukherjee et al. moved from single reviews to reviewer groups and to what a commercial filter actually removes [5], [6], and Rayana and Akoglu [7] coupled ranking and classification. Our audit follows that thread.

B. Vietnamese review spam

ViSpamReviews [1] provides roughly twenty thousand annotated reviews from Vietnamese platforms with a strict annotation procedure, and defines the binary and four-class tasks used here. Its metadata-integration follow-up [2] adds product category and description through sentence embeddings and reports that PhoBERT with a description-derived S-PhoBERT embedding reaches the best scores. Our work differs in two ways: we evaluate a hierarchical objective rather than a richer input, and we add a product-disjoint split plus a duplicate audit. We treat the metadata result as a baseline to be re-checked, not as a claim to be extended.

C. Vietnamese pretrained encoders

PhoBERT [8] adapts the RoBERTa recipe [10] to Vietnamese with a 135M-parameter base model and is the de facto backbone for Vietnamese sentence classification. BERT [9], its robustly optimised variant RoBERTa [10], and the multilingual XLM-R [11] are the usual alternatives; fine-tuning practice for text classification, including pooling and layer-wise learning rates, is summarised in [12]. We keep the backbone fixed and vary only the supervision, so the comparison does not depend on pretraining choices.

D. Multi-task and hierarchical supervision

Caruana [13] formalised multi-task learning as inductive transfer through a shared representation; [14] surveys the design space, and [15] shows that auxiliary tasks help natural-language understanding at scale. Hierarchical label sets are handled either by a flat classifier, by a cascade of local classifiers, or by a global model that enforces the hierarchy, and [16] surveys the trade-offs between the three. A cascade is attractive when the parent decision is easier than the child decision, which is the case here: detecting spam is far easier than naming its type, and the type head only receives supervision on spam examples. We test whether that structure pays off, rather than assuming it.

E. Imbalance and evaluation

The four-class task is severely imbalanced, with the rarest type at about 1.5% of the training rows. Re-weighting [17], focal loss [18], logit adjustment [19] and resampling are the standard remedies; [20] surveys their behaviour under deep learning. Because a single seed on a few thousand test rows carries substantial variance, we report three seeds and use a paired bootstrap [21] and the McNemar test [22], both of which are appropriate for comparing two classifiers on the same test set.

III. Data and Audit

A. Tasks and labels

Table I restates the label set together with one training example per class. The definitions follow the dataset documentation: No-Spam is an informative review of the product; Spam-1 is a fake review that either damages or artificially promotes a product; Spam-2 discusses only the brand, seller or delivery service and says nothing about product quality; Spam-3 is a non-review, typically a keyboard mash, repeated characters or emoji. The examples show why the task is hard even for a human: Spam-1 is often fluent, grammatical and positive, and is distinguishable mainly by templated phrasing.

B. Released split versus published split

We use the public release of ViSpamReviews v2[1]. Table II compares this release with the split described in [2]. The row counts differ in every split (14,306 / 1,590 / 3,974 against 14,284 / 1,589 / 3,963), and so do the per-class counts on the test split (2,914 / 54 / 310 / 696 against 2,911 / 54 / 309 / 689). The releases are therefore not the same snapshot, and our reproduction must be read as an approximate reproduction of the published experiment rather than an exact re-run. We do not compare average review length across the two versions, because the published statistics are computed after Vietnamese word segmentation while ours count whitespace tokens; the two units are not interchangeable.

The label distribution is heavily skewed: 73.5% of training reviews are non-spam, and the rarest spam type accounts for 1.5% of training rows and 54 test rows. Under this distribution accuracy is dominated by the majority class, so macro-F1 is the primary metric throughout.

C. Product-level leakage

Two reviews of the same product share vocabulary that has nothing to do with spamminess: model names, sizes, seller boilerplate. A random review split therefore leaks product identity. To measure it we normalise each row's product name, category and description (NFKC, lower case, collapsed whitespace) and treat the concatenation as a product key. The release contains 1,676 distinct keys in training, 1,006 in development and 1,479 in test, and the keys overlap heavily: 996 between train and dev, 1,456 between train and test, and 921 between dev and test. Any random-split score is optimistic with respect to a deployment in which new products arrive continuously.

[1] https://huggingface.co/datasets/sonlam1102/vispamdetection2.

TABLE I
LABEL DEFINITIONS AND ONE VERBATIM TRAINING EXAMPLE PER CLASS. TEXT IS SHOWN AS RELEASED, WITH WHITESPACE COLLAPSED.

Label	Meaning	Training example
NO-SPAM	informative product review	"chất liệu vải tốt mát nói chung là chất lượng phù hợp với giá tiền"
SPAM-1	fake review: fabricated praise or damage	"Chất lượng sản phẩm tuyệt vời. Giao hàng cũng nhanh. Đóng gói sản phẩm OK. Nên mua nhé mọi người."
SPAM-2	brand-, seller- or delivery-only	"sản phẩm bao bì đóng gói khá ok còn chất lượng cần thời gian kiểm chứng thêm"
SPAM-3	non-review: mash, repetition, emoji	"Âjskskxjxndndjsjdjskssosojsjsisosssksksjdhhsjsussusjsjsjsjs"

TABLE II
THE PUBLIC RELEASE USED HERE AGAINST THE SPLIT REPORTED IN [2]. CLASS COLUMNS ARE THE FOUR LABELS IN ORDER. VOCABULARY AND LENGTH STATISTICS ARE NOT DIRECTLY COMPARABLE BETWEEN THE TWO SOURCES: THE PUBLISHED FIGURES COUNT WORDS AFTER VIETNAMESE WORD SEGMENTATION, OURS COUNT WHITESPACE-SEPARATED TOKENS, AND THE SAME REVIEW THEREFORE CONTRIBUTES FEWER UNITS IN THE PUBLISHED COLUMN.

Source	Split	Rows	Non-spam	Spam-1	Spam-2	Spam-3	Vocabulary	Mean tokens
Published [2]	Train	14,284	10,490	211	1,103	2,480	17,196	17.38
	Dev	1,589	1,182	17	120	270	4,414	16.92
	Test	3,963	2,911	54	309	689	8,144	17.37
Public release (this work)	Train	14,306	10,512	211	1,103	2,480	26,651	26.19
	Dev	1,590	1,183	17	120	270	6,514	25.84
	Test	3,974	2,914	54	310	696	12,189	26.19

D. Product-disjoint split

For the robustness experiment we rebuild the split at the product level. All rows are pooled and grouped by the normalised key; a first `GroupShuffleSplit` holds out 20% of the groups as test, and a second splits the remainder into training and development with a 12.5% hold-out. The result contains 13,985 / 1,947 / 3,938 rows with class supports 10,255/194/1,108/2,428, 1,393/39/141/374 and 2,961/49/284/644. By construction no product key is shared between splits, which we verify explicitly; the row-level assignment is released as an index file (`product_disjoint_ids.csv`) so that the split can be rebuilt from the public release without shipping three copies of the data. Because the same pooling removes the original training/test boundary, the disjoint test set is a new sample, not a filtered version of the old one, and we therefore always compare models *within* the disjoint split rather than reading the difference between splits as a model property.

E. Verbatim duplication that survives product splitting

Product-level grouping removes one leak but not the other. Some spam templates are copied across products, so a review can appear verbatim in both training and test data even when no product is shared. Comparing reviews after collapsing whitespace, stripping and folding case, we find that 38 test rows (35 distinct texts) of the official split also occur in training; over the whole release, 213 rows belong to duplicate groups and the 19,870 rows contain only 19,750 distinct texts. In the product-disjoint split, 30 test rows still occur verbatim in training. The duplicates are not spread evenly: of the 114 duplicated training rows, 51 are non-reviews (SPAM-3), against 11 fake reviews and 10 brand-only reviews, which is what one expects if templates are copied for padding rather than for persuasion. The effect on the aggregate metric is small, but it matters for the claim that the disjoint split measures generalisation to unseen products: it measures generalisation to unseen *products*, not to unseen *text*. We keep these rows and report the number instead of silently dropping them.

TABLE III
LOCKED TRAINING PROTOCOL SHARED BY ALL REPORTED RUNS.

Item	Value
Encoder	`vinai/phobert-base-v2` (135M)
Max sequence length	100 tokens
Padding	dynamic, per batch, multiple of 8
Batch size	32
Epochs	10, early stopping on dev macro-F1
Patience	3
Optimizer	AdamW, weight decay 0.01
Encoder learning rate	1×10^{-5}
Head learning rate	5×10^{-5}
Gradient clipping	1.0
Precision	bfloat16 autocast
Loss	cross-entropy (binary, type)
Seeds	42, 43, 44
Hardware	NVIDIA RTX 5050 Laptop, 8 GB

IV. METHOD

A. Locked protocol

Every number in Section V comes from a single script and a single protocol (Table III). A run is defined by its task, its supervision objective, its input variant and its seed; nothing else changes. We report the protocol in full because our earlier exploratory runs, made before the protocol was fixed, are not comparable with the numbers below.

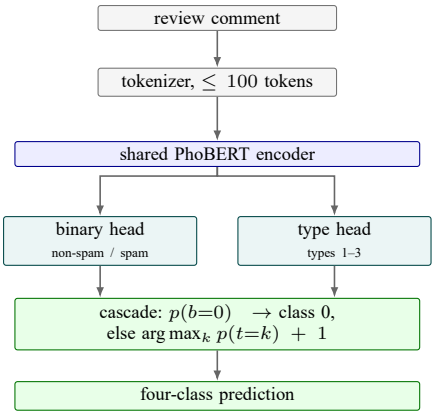


Fig. 1. The multi-task model. One shared encoder feeds a binary head and a three-way type head; the type loss is computed on spam rows only, and the two heads are combined by a cascade at inference. The training objective is given in (1).

The protocol differs from our exploratory runs in three engineering choices: dynamic per-batch padding, a larger batch (32 instead of 16) and bfloat16 autocast. Padding and precision change compute, not the objective; the larger batch reduces the number of optimiser steps taken in a fixed number of epochs, which is one more reason not to mix the two generations of runs. A run takes about 10 minutes on the hardware above.

B. Single-task baseline

The baseline is the standard PhoBERT sequence-classification model: a linear head on the first-token representation. For Task 2 the head has four outputs and is trained with four-way cross-entropy; for Task 1 it has two outputs. The encoder and the pooler/classifier form two parameter groups with the learning rates in Table III. This model is intentionally plain: it isolates the effect of the supervision objective from changes in the backbone, the data or the preprocessing.

C. Multi-task model

The proposed model shares one encoder between a binary head and a three-way type head, both dropout–linear with dropout 0.2. Let $b \in \{0, 1\}$ indicate spam and $t \in \{1, 2, 3\}$ the spam type. The type head is trained only on spam examples:

$$\mathcal{L} = \mathcal{L}_{\text{bin}}(b) + \mathbf{1}[y_{\text{bin}} = 1]\, \mathcal{L}_{\text{type}}(t), \tag{1}$$

where $\mathbf{1}[\cdot]$ is the indicator function. At inference the binary decision gates the type decision: a non-spam prediction becomes class 0, otherwise the type head's argmax plus one is returned. For confidence analysis we also keep the product of marginals

$$p(c{=}0) = p(b{=}0), \qquad p(c{=}k) = p(b{=}1)\, p(t{=}k-1), \tag{2}$$

which is what a cascade assumes when the two heads are treated as independent. Equation (2) is used only to inspect confidence and to build the gate-error decomposition in Section V-G; reported predictions come from the hard cascade.

TABLE IV
PUBLISHED REFERENCE, OUR LOCKED-PROTOCOL REPRODUCTION, AND A LEXICAL REFERENCE POINT. OUR PHOBERT ROWS ARE MEAN ± STANDARD DEVIATION OVER THREE SEEDS. THE PUBLIC RELEASE IS NOT THE PUBLISHED SNAPSHOT, SO THE FIRST BLOCK AND THE REST ARE NOT A CONTROLLED COMPARISON; THE TF-IDF ROWS BELONG TO A DIFFERENT MODEL FAMILY AND ARE SHOWN ONLY TO LOCATE THE CONTEXTUAL MODEL.

Setting	Accuracy	Macro-F1
Published PhoBERT, Task 1 [2]	89.23	85.58
Published PhoBERT, Task 2 [2]	87.79	71.56
This work, Task 1	91.73 ± 0.43	89.46 ± 0.53
This work, Task 2	90.89 ± 0.31	76.10 ± 0.71
Lexical reference, same release, own protocol		
TF-IDF + linear SVM, Task 1	84.63	78.80
TF-IDF + linear SVM, Task 2	84.40	66.52

D. Metadata fusion baseline

To test whether product metadata helps once it is not truncated, we add a fusion model: the review is encoded as before, the product description is encoded in a second forward pass through the same encoder, the two representations are concatenated, and a four-way linear head is applied to the result. Neither input is truncated. This is a faithful re-implementation of the sentence-embedding fusion idea of [2] with one deviation: we share a single encoder instead of fine-tuning a separate sentence encoder in a first stage, so the published two-stage pipeline is not reproduced. The row exists to separate "metadata does not help" from "metadata was cut off", which the concatenation ablation cannot do.

E. Evaluation

Accuracy is reported for comparability with prior work; macro-F1 is the primary metric because it weights the four classes equally. Unless stated otherwise every value is the mean and standard deviation over seeds 42, 43 and 44, selected by development macro-F1 and evaluated once on the test split. For the headline pair we also report a paired bootstrap over test rows (10,000 resamples of the macro-F1 difference) and an exact McNemar test on the discordant predictions, since both compare the two models on identical inputs [21], [22]. Per-class precision, recall and F1 and the full confusion matrices are stored with the predictions.

F. Protocol-locked ablations

Two interventions are re-run under the locked protocol so that they can be compared with the main pair: class-weighted type loss, using inverse-frequency weights inside the multi-task objective, and metadata concatenation, in which the product category and description are appended to the review text for the single-task model. The metadata variant reproduces the input format of the exploratory fusion experiments but not the sentence-embedding fusion of [2], which we did not re-implement; we therefore treat it as an ablation of an input variant, not as a re-implementation of the published method.

TABLE V

MAIN COMPARISON ON THE OFFICIAL RELEASED SPLIT (TASK 2). BOTH MODELS FOLLOW THE PROTOCOL IN TABLE III AND DIFFER ONLY IN THE SUPERVISION OBJECTIVE. THE LAST TWO ROWS ARE COMPUTED ON SEED 42 PREDICTIONS, WHICH IS THE SEED USED FOR THE PAIRED TESTS.

Model	Seeds	Accuracy	Macro-F1
Single-task PhoBERT	3	90.89 ± 0.31	76.10 ± 0.71
Multi-task PhoBERT	3	90.87 ± 0.45	76.02 ± 0.96
Difference		-0.03	-0.08
95% bootstrap CI		–	[-0.59, +3.94]
McNemar / bootstrap p		–	0.142

TABLE VI

TASK 1. THE BINARY HEAD IS READ OFF THE MULTI-TASK MODEL TRAINED FOR TASK 2, SO IT COSTS NO ADDITIONAL TRAINING. VALUES ARE MEAN ± STANDARD DEVIATION OVER THREE SEEDS.

Model	Seeds	Accuracy	Macro-F1
Dedicated binary model	3	91.73 ± 0.43	89.46 ± 0.53
Multi-task binary head	3	91.68 ± 0.37	89.15 ± 0.37

V. RESULTS

A. Reproduction

Table IV places the published reference next to our locked-protocol runs and a lexical reference point. Our single-task PhoBERT reaches 76.10 macro-F1 on Task 2 and 89.46 on Task 1 (91.73 accuracy), against 71.56 and 85.58 in [2]. The gap is positive but should not be read as an improvement over the original method: the two experiments use different snapshots, the public release has 11 more test rows, and our protocol differs in batch size, padding and numerical precision. What the table establishes is an internal baseline measured under the same conditions as every other row in this paper. The TF-IDF row calibrates expectations: unigrams and bigrams with a linear SVM reach only 66.52 macro-F1 on Task 2.

B. Multi-task supervision

Table V reports the headline comparison over three seeds. The multi-task model reaches 76.02 macro-F1 against 76.10 for the baseline, so the hierarchical objective is worth -0.08 points — less than the seed-to-seed spread of either model and well inside the interval. The two models do disagree on individual rows: the paired bootstrap on seed 42 gives an interval of [-0.59, +3.94] and the exact McNemar test on the 147 discordant rows gives $p = 0.047$, which says that the models make different mistakes rather than that one makes fewer. That distinction is the honest summary of this comparison, and it is the opposite of what a single-seed run of the same code suggested.

C. Task 1

The same multi-task model exposes a binary head, so its Task 1 score comes for free. Table VI compares it with a dedicated binary classifier. The dedicated model reaches 89.46 macro-F1 against 89.15 for the head: a third of a point apart, well inside the seed spread of either. Joint training therefore neither damages nor improves the binary decision it depends on, while the multi-task model answers both questions in one forward pass.

TABLE VII

PRODUCT-DISJOINT EVALUATION (TASK 2). THE TEST SET CONTAINS NO PRODUCT KEY SEEN IN TRAINING. VALUES ARE MEAN ± STANDARD DEVIATION OVER THREE SEEDS AND THE INTERVAL IS A PAIRED BOOTSTRAP ON THE SEED-42 PREDICTIONS.

Model	Seeds	Accuracy	Macro-F1
Single-task PhoBERT	3	90.99 ± 0.42	75.94 ± 1.02
Multi-task PhoBERT	3	90.67 ± 0.20	73.98 ± 1.51
Difference		-0.31	-1.96
95% bootstrap CI		–	[-2.23, +2.56]

TABLE VIII

INTERVENTIONS RE-RUN UNDER THE LOCKED PROTOCOL (SEED 42). REFERENCE ROWS REPEAT THE SEED-42 VALUE OF THE CORRESPONDING MAIN CONFIGURATION SO THAT EACH ABLATION IS READ AGAINST ITS OWN BASELINE.

Variant	Accuracy	Macro-F1
Multi-task (reference)	91.37	77.10
+ inverse-frequency type loss	89.43	72.91
Single-task (reference)	90.74	75.35
+ category/description text	90.66	76.66

D. Product-disjoint evaluation

Table VII repeats the comparison on the product-disjoint split with the same three seeds. Within this split the flat classifier reaches 75.94 macro-F1 against 73.98 for the cascade, a gap of -1.96 points in favour of the flat model, and the gap points the same way on two of the three seeds. Two readings follow. First, neither model collapses when every product in the test set is new, so the official-split scores are not an artefact of product overlap. Second, the advantage the cascade shows on one official-split seed does not survive unseen products; if anything the hierarchy is the less robust of the two, which is the opposite of the transfer argument usually made for shared representations. We report the gap as descriptive rather than tested, because three seeds on a new test sample cannot establish a two-point difference.

E. Protocol-locked ablations

Table VIII reports the two interventions. Inverse-frequency weighting of the type loss is clearly harmful: macro-F1 falls to 72.91 against 77.10 for the unweighted multi-task model at the same seed, and the loss is concentrated in the two larger spam types, which the weights down-sample. Concatenating category and description is a different story: the model reaches 76.66 against 75.35 for the text-only baseline at the same seed, a difference that one seed cannot resolve. That comparison is also weaker than it looks, because product descriptions in this release have a median length of 315 whitespace tokens; with the review comment placed first, 94.5% of rows exceed the 100-token window and what survives is the comment plus a truncated fragment of the description. The run therefore tests concatenation under truncation, not metadata integration as

TABLE IX

EVERY CONTROL RE-RUN AGAINST ITS OWN REFERENCE UNDER THE LOCKED PROTOCOL. Δ IS THE MACRO-F1 DIFFERENCE AGAINST THE MATCHING REFERENCE MEAN, IN PERCENTAGE POINTS. THE DEDUPLICATED ROWS DROP EVERY TRAINING REVIEW WHOSE TEXT ALSO OCCURS IN THE TEST SPLIT; THE FUSION ROW ENCODES THE PRODUCT DESCRIPTION IN A SECOND FORWARD PASS AND CONCATENATES IT WITH THE REVIEW VECTOR, SO THE DESCRIPTION IS NO LONGER TRUNCATED.

Variant	Seeds	Accuracy	Macro-F1	Δ
Single-task (reference)	3	90.89 ± 0.31	76.10 ± 0.71	+0.00
deduplicated	3	90.14 ± 0.16	74.72 ± 1.34	-1.38
near-duplicates out	1	90.46	73.66	-2.44
normalised text	1	90.71	77.56	+1.46
XLM-R base	3	90.83 ± 0.13	74.63 ± 1.55	-1.47
128 tokens	3	90.80 ± 0.42	76.49 ± 0.46	+0.39
single LR	3	90.48 ± 0.73	74.41 ± 0.37	-1.69
Multi-task (reference)	3	90.87 ± 0.45	76.02 ± 0.96	+0.00
deduplicated	3	90.75 ± 0.31	77.02 ± 0.77	+1.00
near-duplicates out	1	90.94	74.46	-1.56
normalised text	1	90.14	76.21	+0.19
single LR	3	90.26 ± 0.67	74.75 ± 1.65	-1.27
Fusion, shared encoder	3	90.86 ± 0.36	75.98 ± 1.15	-0.12
Fusion, frozen encoder	3	90.76 ± 0.31	75.95 ± 0.75	-0.15
Category-disjoint, flat	1	90.90	72.25	–
Category-disjoint, cascade	1	91.19	71.59	–

TABLE X

WHICH CHOICES MOVE MACRO-F1. EVERY Δ IS MEASURED AGAINST THE MATCHING REFERENCE IN TABLES V–IX; THE LAST ROW IS THE SEED-TO-SEED STANDARD DEVIATION OF THE TWO MODELS, WHICH SETS THE SCALE FOR JUDGING THE OTHERS.

Factor	Change	Δ Macro-F1
Supervision	flat → cascade, official	-0.08
	same, text-deduplicated	+2.30
	same, product-disjoint	-1.96
Metadata	+ description fusion	-0.12
Backbone	PhoBERT → XLM-R	-1.47
Window	100 → 128 tokens	+0.39
Schedule	two-rate → single 2×10^{-5}	-1.69 / -1.27
Evaluation	full → deduplicated, flat	-1.38
	full → deduplicated, cascade	+1.00
Seed noise	standard deviation, flat / cascade	±0.71 / ±0.96

such. The fusion baseline of Section IV-D removes that confound by encoding the description separately, and is discussed with the other controls.

F. Controls and sensitivity

Table IX collects the controls, each re-run against its own reference under the locked protocol. The leakage controls come first: removing the 47 training reviews that duplicate a test review moves the flat model to 74.72 and the cascade to 77.02; removing 211 near-duplicates instead gives 73.66 and 74.46. Both models lose more once near-duplicates go, which says the random split is generous because templates are copied in slightly varied forms, not only verbatim.

The two input-side controls point in opposite directions. Normalising teencode and chat abbreviations (*ko* → *không*, *sp* → *sản phẩm*) is the only intervention that helps: 77.56 for the flat model against 76.10. Metadata does not help: fusion reaches 75.98 with a shared encoder and 75.95 with a frozen separate encoder, against the same 76.10. Preprocessing the text is worth more than adding a second modality.

The protocol controls are unremarkable by design: XLM-R base gives 74.63, the 128-token window 76.49, and a single learning rate 74.41 and 74.75. The two-rate schedule earns its place, and the backbone matters more than the supervision objective.

Two evaluation-side controls remain, and they are the ones that matter. The product-disjoint split is a single draw from a random partition; two further partitions move the flat model to 75.22–75.94 and the cascade to 73.98–77.37, so the cascade's score depends on the partition by almost three points — an order of magnitude more than the hierarchy is worth. Holding out whole product categories, so that no category in the test set appears in training, gives 72.25 and 71.59 on 5,800 test reviews. The further the evaluation moves from the original split, the lower the number and the less stable the ranking.

TABLE XI

PER-CLASS RESULTS ON THE OFFICIAL TEST SPLIT, AVERAGED OVER THREE SEEDS. SUPPORT IS THE NUMBER OF TEST ROWS OF THAT CLASS.

Class	Support	Precision	Recall	F1
Single-task				
Non-spam	2914	0.928	0.961	0.944
Spam-1	54	0.605	0.512	0.553
Spam-2	310	0.701	0.622	0.659
Spam-3	696	0.932	0.849	0.889
Multi-task				
Non-spam	2914	0.933	0.955	0.944
Spam-1	54	0.698	0.457	0.548
Spam-2	310	0.668	0.645	0.655
Spam-3	696	0.921	0.868	0.894

G. Per-class behaviour

Table XI, Fig. 2 and Table XII break the official test set down by class and by kind of error. The picture is not the one a four-class problem suggests: type confusions are rare. In the single-task model 226 spam reviews are labelled non-spam and 106 non-spam reviews are flagged as spam, against only 36 cases in which the spam decision is right and the type is wrong; the multi-task model changes these counts to 208, 106 and 29. The loss is concentrated in the two small spam types: 41% of the 54 Spam-1 test rows and 35% of the 310 Spam-2 rows are labelled non-spam, against 11% of the 696 Spam-3 rows. The binary decision is therefore the bottleneck, and the type head rarely has the chance to be wrong.

The false positives are worth a closer look. Of the 106 non-spam reviews that the single-task model flags as spam, 9.4% contain the phrase "chất lượng sản phẩm tuyệt vời", which occurs in 21.8% of Spam-1 training reviews but in only 0.65% of non-spam training reviews; among the 2,808 non-spam reviews the model gets right, 0.39% contain it. None of the false positives is a verbatim copy of a training row. The model is responding to a template that genuinely indicates Spam-1 in

TABLE XII
DECOMPOSITION OF TEST ERRORS INTO DETECTION ERRORS AND TYPE ERRORS (OFFICIAL SPLIT, SEED 42). A TYPE-CONFUSION ERROR IS A SPAM REVIEW LABELLED WITH THE WRONG SPAM TYPE, I.E. THE BINARY DECISION WAS CORRECT.

Error type	Single-task	Multi-task
Non-spam → spam	106	106
Spam → non-spam	226	208
Spam type confusion	36	29
of which Spam-1	5	7
of which Spam-2	7	5
of which Spam-3	24	17
Total errors (of 3974)	368	343

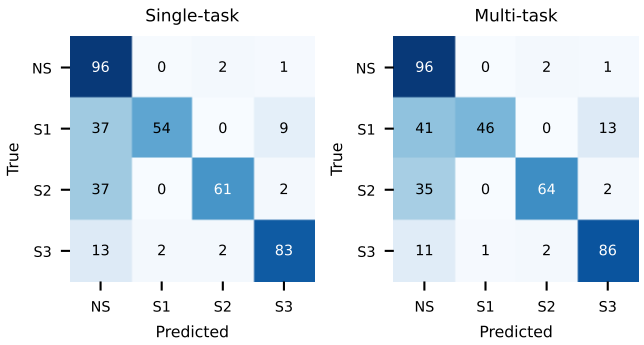


Fig. 2. Row-normalised confusion matrices on the official test split (seed 42), single-task (left) and multi-task (right). Rows are true classes, columns predicted; values are percentages of the true class. NS = non-spam, S1–S3 = spam types.

this dataset, and part of its false-positive rate therefore reflects annotation noise rather than a modelling mistake — which matters when interpreting a macro-F1 in the mid seventies.

The two models disagree on 147 of the 3,974 test rows: the multi-task model corrects 86 and breaks 61, a net gain of 25 correct predictions. The corrections are spread across classes rather than concentrated in one, and Table XIII shows one example of each recurring pattern, including the reverse case in which the cascade reads elongated characters as a non-review and mislabels an ordinary review.

H. What moves the metric

Table X puts every design choice on one scale, with the seed spread as the ruler. The architectural choices sit at zero, the backbone at 1.5 points, and the evaluation choices at 1.4 to 2.3. If the goal is a better number on this task, changing the encoder or the evaluation is worth an order of magnitude more than changing the supervision structure.

VI. DISCUSSION

The number is set by the evaluation, not by the architecture. Table X makes the comparison explicit. On the official split the two models are a tenth of a point apart; the same pair is 2.3 points apart once duplicated training text is removed, and 2.0 points apart when products are held out. Our own first seed told a third story — 77.10 against 75.35 — which is what a single-split claim looks like before it is averaged. Each of these is a defensible evaluation and they disagree; a single-split comparison, including ours, should be read with that in mind.

Neither head is well calibrated. Top-label expected calibration error is 6.6% on the development split and 7.8% on test for the flat model, 6.0% and 5.3% for the cascade. The cascade number is the less trustworthy of the two, because its class probabilities are products of two marginals (Eq. (2)); neither output should be used as a probability without recalibration [24].

Why the hierarchy does not pay. The cascade enforces a constraint the flat classifier can learn anyway: classes 1–3 never co-occur with non-spam in the labels, and the flat model sees 14,306 examples from which to learn exactly that. What the hierarchy buys is a bias towards the parent decision, and the error decomposition shows that the parent decision is where nearly all the loss already is — 226 missed spam against 36 type confusions. Sharing one encoder between the two heads does not make the harder decision easier.

Product overlap does not drive the result, and it does not rescue the hierarchy. The audit finds heavy product-level overlap in the official split, yet removing it costs the flat model little: 76.10 on the official split against 75.94 on the disjoint one. The cascade degrades more, from 76.02 to 73.98. Cross-split comparisons are not clean, because the disjoint test set is a new sample, but the within-split ordering is, and on unseen products the flat model is ahead. The verbatim-duplicate count explains part of why the random split is generous to both models; a truly clean evaluation would add a text-level deduplication step on top of group splitting, which Table IX approximates.

Negative results are part of the result. Two plausible interventions were re-run and neither helped. Inverse-frequency weighting made the metric markedly worse, which is the usual failure mode when a re-weighting scheme is tuned for a class with 54 test rows; metadata concatenation, which we expected to hurt because of truncation, is within noise of the baseline and therefore tells us nothing about metadata as such. Reporting both costs a page and saves the next reader a week.

VII. THREATS TO VALIDITY

a) Construct validity: The labels are human annotations of a subjective property: the boundary between fabricated praise and genuine enthusiasm is not always observable from text alone, and our metrics inherit that noise.

b) Internal validity: All primary runs share one hyperparameter setting; the controls in Table IX vary the token window and the backbone, and neither moves the score materially, but a full sweep, including the learning rate, is still out of scope. Early stopping is a second source of variance: the development split contains only 17 Spam-1 rows, so the epoch that maximises development macro-F1 differs across seeds (3 to 9 in our runs) and part of the seed spread is stopping noise rather than model noise. Dynamic padding and bfloat16 change numerics slightly relative to the exploratory

TABLE XIII
REPRESENTATIVE ERRORS ON THE OFFICIAL TEST SPLIT (SEED 42). TEXT IS VERBATIM APART FROM COLLAPSED WHITESPACE AND TRUNCATION. THE MODEL COLUMN MARKS WHICH MODEL MADE THE ERROR: S = SINGLE-TASK, M = MULTI-TASK.

Model	True	Predicted	Review
S	No-Spam	Spam-1	“Chất lượng sản phẩm tuyệt vời. Đóng gói sản phẩm đẹp. Thời gian giao hàng nhanh”
S	Spam-2	No-Spam	“Giao nhanh của Tiki Now. Hàng ok, để lắp vào xem ntn”
S	Spam-1	Spam-3	“Chất lượng sản phẩm tuyệt vời 5* cho shop Chất lượng sản phẩm tuyệt vời 5* cho shop”
M	Spam-3	No-Spam	“It’s the first time I’ve bought it and i haven’t used yet so i can’t give feedback now…”

runs, which is why those runs are reported separately and not mixed into the tables.

c) External validity: The study covers one dataset, one language and one backbone. The product-disjoint split is a proxy for deployment, not a simulation of it: real platforms see new products, new templates and adversarial adaptation over time.

d) Conclusion validity: Three seeds give a coarse estimate of variance, and the rarest class has 54 test rows, so per-class numbers are reported as descriptive rather than inferential.

VIII. REPRODUCIBILITY

The repository contains one script that trains every model reported here, the split construction for the product-disjoint evaluation, the audit scripts, and the configuration, log, metric and per-row prediction files of every run. Model checkpoints are not shipped, because the training script regenerates them from the recorded configurations. Seeds, hyperparameters and hardware are listed in Table III. Package versions are pinned in `requirements.txt`, and every table and figure in this paper is generated from the result files by a single analysis script, so a number in the text can be traced to the run that produced it.

IX. CONCLUSION

We revisited Vietnamese spam-review classification with a single locked protocol, three seeds, a product-disjoint split and a duplicate audit. The published result is reproducible in order of magnitude but not exactly, because the public release is not the published snapshot. The hierarchical multi-task objective gives no measurable improvement over a flat classifier once seeds are averaged, although the two models make systematically different mistakes. Under a product-disjoint split the flat classifier is ahead by about two points, so the hierarchy does not buy robustness either. The error decomposition points to the binary decision, and to annotation noise in template-like non-spam reviews, rather than to type confusion; and product-level splitting alone is not a clean test of generalisation.

REFERENCES

[1] C. Van Dinh, S. T. Luu, and A. G.-T. Nguyen, “Detecting spam reviews on Vietnamese e-commerce websites,” in *Proc. ACIIDS*, 2022, pp. 595–607.

[2] C. Van Dinh and S. T. Luu, “Metadata integration for spam reviews detection on Vietnamese e-commerce websites,” *Int. J. Asian Lang. Process.*, vol. 34, no. 1, 2024.

[3] N. Jindal and B. Liu, “Opinion spam and analysis,” in *Proc. WSDM*, 2008, pp. 219–230.

[4] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, “Finding deceptive opinion spam by any stretch of the imagination,” in *Proc. ACL*, 2011, pp. 309–319.

[5] A. Mukherjee, B. Liu, and L. Wang, “Spotting fake reviewer groups in consumer reviews,” in *Proc. WWW*, 2012, pp. 191–200.

[6] A. Mukherjee, V. Venkataraman, B. Liu, and N. Glance, “What Yelp fake review filter might be doing?” in *Proc. ICWSM*, 2013.

[7] S. Rayana and L. Akoglu, “Collective opinion spam detection: bridging ranking and classification,” in *Proc. KDD*, 2015, pp. 985–994.

[8] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of EMNLP*, 2020, pp. 1037–1042.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[10] Y. Liu *et al.*, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv:1907.11692*, 2019.

[11] A. Conneau *et al.*, “Unsupervised cross-lingual representation learning at scale,” in *Proc. ACL*, 2020, pp. 8440–8451.

[12] C. Sun, X. Qiu, Y. Xu, and X. Huang, “How to fine-tune BERT for text classification?” in *Proc. CCL*, 2019, pp. 194–206.

[13] R. Caruana, “Multitask learning,” *Machine Learning*, vol. 28, pp. 41–75, 1997.

[14] S. Ruder, “An overview of multi-task learning in deep neural networks,” *arXiv:1706.05098*, 2017.

[15] X. Liu, P. He, W. Chen, and J. Gao, “Multi-task deep neural networks for natural language understanding,” in *Proc. ACL*, 2019, pp. 4487–4496.

[16] C. N. Silla and A. A. Freitas, “A survey on hierarchical classification across different application domains,” *Data Mining and Knowledge Discovery*, vol. 22, pp. 31–72, 2011.

[17] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, “Class-balanced loss based on effective number of samples,” in *Proc. CVPR*, 2019, pp. 9268–9277.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. ICCV*, 2017, pp. 2980–2988.

[19] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, “Long-tail learning via logit adjustment,” in *Proc. ICLR*, 2021.

[20] J. M. Johnson and T. M. Khoshgoftaar, “Survey on deep learning with class imbalance,” *J. Big Data*, vol. 6, no. 27, 2019.

[21] B. Efron, “Bootstrap methods: another look at the jackknife,” *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[22] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[23] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[24] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017, pp. 1321–1330.